

SPEC-001-Personal Secure Web Gateway

Background

Modern end users frequently browse the internet without visibility into the security posture of visited websites, downloaded content, or credential usage. Common risks include unsafe browsing, malicious links, credential reuse across services, and lack of behavioral insight into personal network activity. Traditional enterprise security controls (secure gateways, SIEM monitoring, Zero Trust verification) are typically unavailable to individuals. This project addresses these risks by designing a personal secure web gateway that intermediates all browser traffic, performs automated reputation checks, securely manages credentials locally, and provides continuous visibility and verification for a single user environment.

This project designs and implements a personal secure web gateway firewall intended for a single end user within a VMware laboratory environment. The system acts as an intermediary between the user and the internet, ensuring that all web traffic passes through a controlled security gateway before reaching external services.

Unlike manual scanning tools, the platform performs automated inspection, monitoring, and decision-making transparently in the background. The goal is to demonstrate realistic cybersecurity architecture concepts while remaining implementable by a single student developer.

The platform combines multiple defensive concepts into a modular personal security ecosystem:

- Secure proxy firewall controlling outbound traffic
- Automated URL and simulated file reputation scanning
- Local encrypted password manager
- Traffic metadata collection and behavioral analysis
- Personal SIEM-style logging and visualization
- Zero Trust verification principles

Deployment consists of two VMware virtual machines:

1. User VM – browser and Angular dashboard
2. Gateway VM – NestJS security platform and database

Requirements

Functional Requirements

MUST HAVE (MVP)

- Explicit proxy gateway intercepting browser traffic
- URL reputation scanning via VirusTotal API
- Simulated file download scanning
- Allow/Block policy engine
- Master-password authentication
- Local encrypted password vault
- Traffic metadata logging
- Angular dashboard with live updates
- NestJS backend (modular architecture)
- PostgreSQL database (Docker container)
- WebSocket-based live log streaming
- Custom firewall block page

SHOULD HAVE (Phase 2)

- Behavioral baseline detection
- UEBA-style anomaly scoring
- Threat notifications
- Modular event-driven communication

COULD HAVE (Future Enhancements)

- Password generation automation
- Multi-device component distribution
- Advanced analytics dashboards

WON'T HAVE (Scope Control)

- HTTPS MITM inspection
- Kernel-level firewalling
- Enterprise antivirus engines

Security Requirements

The system shall enforce measurable security guarantees aligned with Zero Trust principles.

Authentication and Access Control

- All sensitive operations require authenticated sessions
- Continuous verification using session freshness checks
- Re-authentication required after defined inactivity period
- Sessions bound to device fingerprint (IP + User-Agent)

Cryptography and Data Protection

- Password hashing using Argon2id
- Encryption-at-rest for vault credentials using AES-256-GCM
- Encryption keys derived from master password via KDF
- Encryption keys never stored persistently

Zero Trust Principles

- No implicit trust between components
- Every request validated before processing
- Session validity checked per request

Audit and Observability

- All proxy decisions logged
- Security-relevant events stored in audit logs
- Traffic metadata collected for analysis
- Real-time monitoring via WebSocket event stream
- URL reputation scanning via VirusTotal API
- Master-password authentication
- Local encrypted password vault
- Traffic metadata logging
- Zero Trust session verification

Method

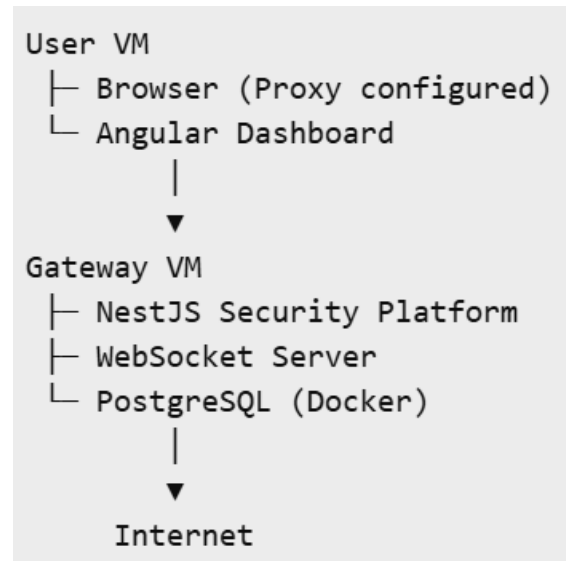
System Architecture Overview

The proxy operates at the HTTP application layer (OSI Layer 7). It analyzes and mediates web requests at the protocol level rather than performing packet-level inspection or acting as a network firewall. System focuses on application-layer security controls such as request validation, reputation checks, authentication enforcement, and logging, while excluding lower-layer packet inspection or kernel-level filtering.

All traffic is mediated through a central proxy. All security validation occurs before traffic is forwarded:

Browser → Gateway Proxy → Internet

VMware Network Topology



Backend Architecture (NestJS Monolith with Modules)

Modules:

- ProxyModule – traffic interception and forwarding
- AuthModule – authentication and Zero Trust checks
- ThreatModule – external reputation scanning
- VaultModule – password manager
- SIEMModule – logging and analytics

Internal communication uses an event-driven model via NestJS EventEmitter.

A modular monolith architecture was intentionally selected instead of microservices due to security focus over infrastructure complexity. NestJS modules provide strong boundaries similar to microservices while remaining within a single deployable application. Modules can later be extracted into independent services for distributed deployment.

Request Processing Pipeline

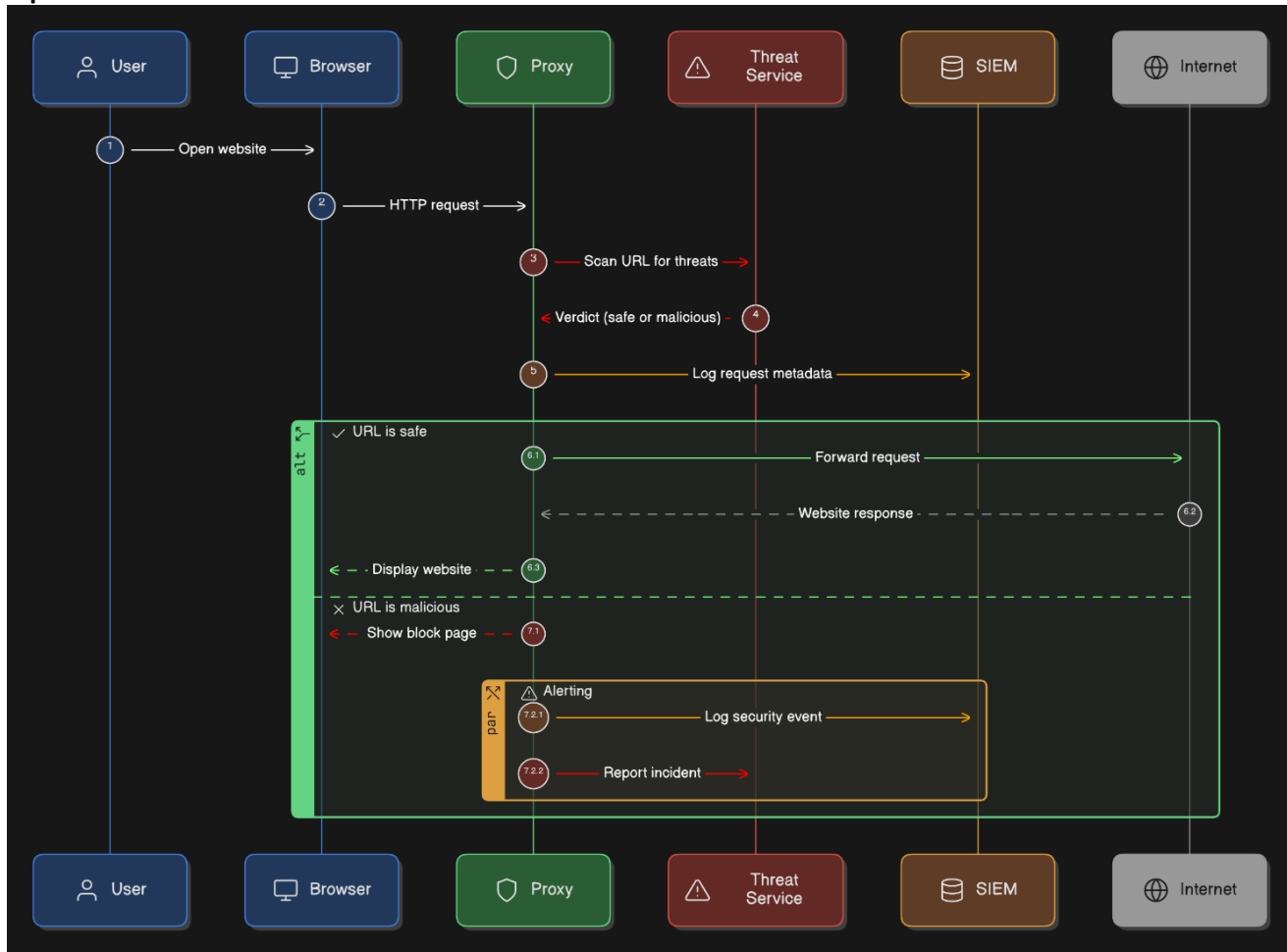
Failure Handling Policy

Security decisions must explicitly define behavior when external dependencies fail. The gateway follows a fail-safe but usable model:

- VirusTotal API timeout or unavailable → Request allowed with *"Unverified"* warning logged and risk score increased.
- Scanning service error response → Default allow with security event logged for audit visibility.
- Internal proxy error → Request blocked and custom error page returned.
- Database unavailable → Sensitive operations (authentication, vault access) denied; proxy continues operating in limited logging mode.
- Authentication verification failure → Request blocked and session invalidated.

All failure events are recorded in `security_events` for later analysis and displayed in the dashboard.

Pipeline



Zero Trust Enforcement

No request is implicitly trusted regardless of network location. This system applies Zero Trust principle by requiring verification for every interaction, independent of whether traffic originates from the local network, trusted VM, or previously authenticated session.

Each request verifies:

- Valid JWT session
- Active session record
- Device fingerprint (IP + User-Agent)
- Recent authentication timestamp

Sensitive actions require re-authentication:

If $\text{now} - \text{last_auth_at} > 20 \text{ minutes}$: require password conformation

Cryptography Model

Threat Model Assumptions

The cryptographic design is based on the assumptions of what the attacker may do: gain access to the database storage or observe encrypted traffic between components. Attacker does not have access to runtime memory of the gateway process while the user is authenticated and does not know the user's master password.

Security goals under these assumptions:

- Database compromise must not reveal stored credentials.
- Captured traffic must remain confidential.
- Encryption keys exist only in RAM memory and are re-derived after authentication.

Authentication

Password hashing: Argon2id
Session tokens: JWT in HttpOnly Secure cookies

Vault Encryption

<u>Key derivation:</u>	<u>Encryption algorithm:</u>	<u>Stored values:</u>
Master Password + User Salt	AES-256-GCM	- Ciphertext
↓		- IV
Argon2 KDF		- Authentication tag
↓		
Vault Encryption Key (memory only)		

Database Schema

Performance and Indexing Strategy

Because dashboard analytics and real-time monitoring rely heavily on querying log data, database indexing is required to maintain performance. The traffic_logs table is expected to be the largest dataset and therefore includes optimized indexes:

- Index on timestamp → enables fast time-range queries for dashboards.
- Composite index on (user_id, timestamp) → supports per-user activity timelines.
- Index on verdict → accelerates filtering by allowed/blocked traffic.
- Optional index on risk_score → improves analytics and anomaly views.

Entities

users	sessions	vault_credentials	traffic_logs	security_events
- id (UUID) - username - password_hash - created_at - last_auth_at	- id - user_id (FK) - jwt_id - ip_address - user_agent - created_at - expires_at - last_verified_at	- id - user_id (FK) - service_name - username - encrypted_password - iv - created_at - updated_at	- id - user_id (FK) - url - method - verdict - risk_score - timestamp	- id - type - severity - message - metadata (JSONB) - created_at

Dashboard Architecture

Angular application provides:	Communication:
• Live traffic monitoring • Threat history • Password vault UI • Security analytics view	REST → authentication and CRUD WebSocket → live SIEM event stream

Event-Based WebSocket Design

WebSocket communication follows an event-driven messaging model to simplify frontend subscription handling and RxJS stream composition. Each message contains an explicit event type allowing the dashboard to react selectively.

Event types:

- TRAFFIC_LOG — emitted when a proxied request is processed
- SECURITY_EVENT — emitted for warnings, failures, or detected threats
- SESSION_EVENT — authentication or re-verification updates
- SYSTEM_STATUS — gateway health or dependency status

Angular services map events into RxJS streams:

- traffic\$ → filters TRAFFIC_LOG
- securityEvents\$ → filters SECURITY_EVENT
- systemStatus\$ → filters SYSTEM_STATUS

This approach enables scalable real-time UI updates while keeping frontend logic modular and aligned with the backend event architecture. RxJS Observables power real-time updates.

Implementation

Phase 1 – Infrastructure

1. Create VMware network
2. Configure User VM proxy settings
3. Setup Gateway VM
4. Deploy PostgreSQL via Docker
5. Initialize NestJS project

Phase 2 – Core Backend

1. Implement AuthModule (Passport + JWT)
2. Implement Session tracking
3. Build ProxyModule request interceptor
4. Integrate VirusTotal API
5. Create block page response

Phase 3 – Data & Logging

1. Prisma schema implementation
2. Traffic logging
1. SIEM event emitter
2. WebSocket gateway

Phase 4 – Password Vault

1. Argon2 key derivation
2. AES-256-GCM encryption
3. Credential CRUD APIs
4. Re-authentication guard

Phase 5 – Frontend

1. Angular dashboard setup
2. Authentication flow
3. Live logs using RxJS
4. Vault UI

Gathering Results

Evaluation criteria:

- All browser traffic routed through gateway
- Malicious URLs blocked correctly
- Logs generated for each request
- Vault credentials encrypted at rest
- Re-authentication enforced
- Dashboard displays real-time activity

Performance observations:

- Proxy latency overhead
- Scan response time
- WebSocket throughput
- Resource usage on Gateway VM

Success is achieved when the system demonstrates continuous verification, secure credential handling, and observable security monitoring within the lab environment.